SQL Workshop Outline

Workshop Overview:

- Introduction to SQL (What, Why, How)
- Relational and Non-relational Data (Database)
- Installation Guide and Recommended Learning Resources (Types of SQL)
- Data Schema: Preview the data types for the table
- `SELECT` Statement
- `DISTINCT` Clause
- `LIMIT` and `OFFSET` Clause
- `WHERE` Clause with Wildcards
- SQL Built-in Functions (Aggregate Functions, Scalar Functions, and Other Functions)
- `ORDER BY` Clause
- `GROUP BY` and `HAVING`
- Operators (Logical, `LIKE` `IN` `NOT`, etc.)
- SQL JOINs (`INNER JOIN` `LEFT JOIN` `RIGHT JOIN` `FULL OUTER JOIN`)
- Summary
- Exercises on HackerRank or LeetCode if time permits

What is SQL?

SQL stands for Structural Query Language. It is the standard programming language used to communicate with relational database management systems. You can think of SQL as a specialized translator between you and your data. When you want information—like finding a specific customer or calculating total sales—you cannot simply speak English to the computer. Instead, you write a query in SQL.

Why are we learning SQL?

Imagine you have a spreadsheet with more than 1 million rows; if you try to open that in Excel, it crashes. In the financial world, datasets rarely fit into a single spreadsheet. SQL is the fundamental skill that allows you to break through this barrier and handle massive amounts of financial, accounting, and business data efficiently. It is currently the number one technical skill requested for analyst roles outside of Excel, making it a critical differentiator for your resume.

What can we do with SQL?

SQL empowers you to move beyond simple data entry into true data analysis. In this one-day, hands-on workshop, you will gain practical experience in querying and analyzing data directly, allowing you to ask complex questions and get immediate answers without relying on IT support. Beyond direct analysis, SQL is the "engine" that powers modern business intelligence tools.

Below are some examples of dashboards that demonstrate the powerful capabilities of integrating SQL with SAS.

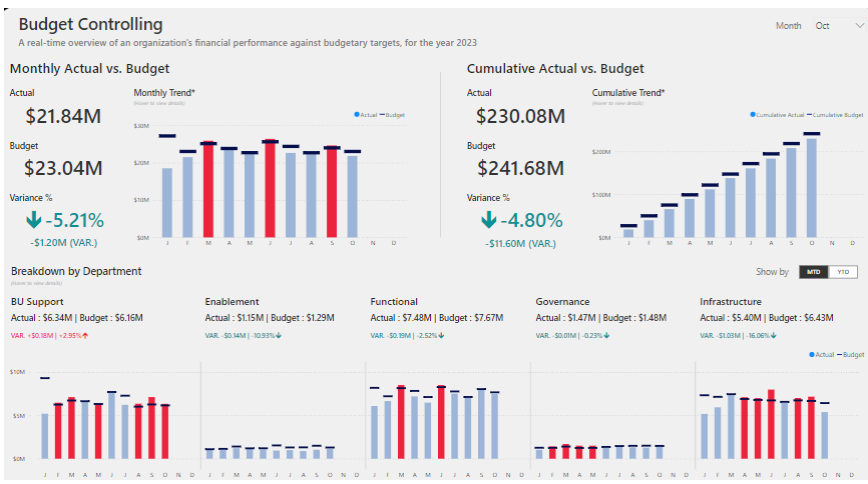


Image retrieve from google image search, Marketing Dashboard example

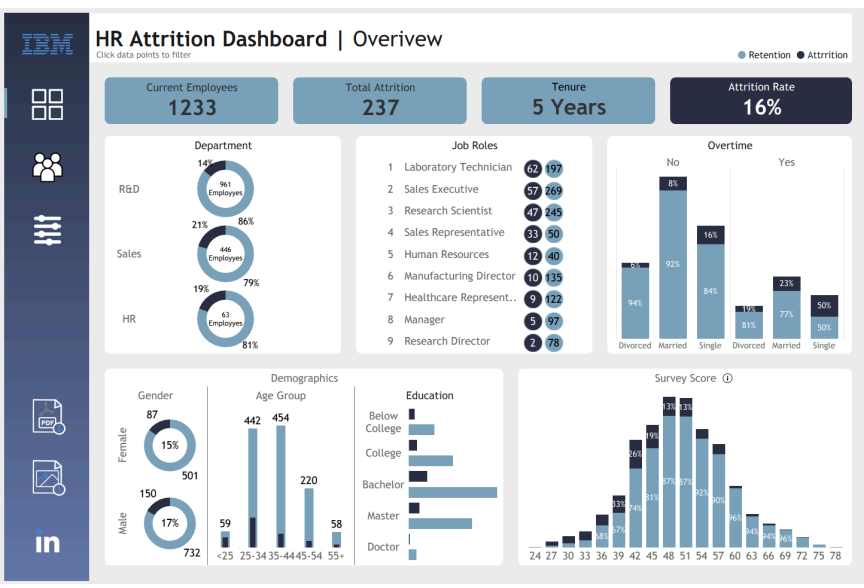


Image retrieve from google image search, IBM Dashboard example

What type of SQL languages and databases are there?

Types of SQL Languages (The "Dialects")

Open-Source

Database	Notes	Dialect Example
MySQL	Very popular, web apps, fast reads	MySQL SQL
PostgreSQL	Advanced, highly standards-compliant, loved in data/finance	PostgreSQL SQL
SQLite	Lightweight, file-based, embedded systems	
MariaDB	MySQL fork, better performance in some cases	

Commercial / Enterprise

Database	Notes	Dialect Example
Oracle DB	Extremely powerful, expensive, enterprise	**PL/SQL**
Microsoft SQL Server	Tight Windows + Azure integration	**T-SQL**
IBM Db2	Banking, insurance, mainframes	
SAP HANA	In-memory, analytics-heavy	

Roughly 90% of the syntax. especially the core `SELECT`, `FROM`, and `WHERE` commands, is identical across all of them. If you learn standard SQL, you can adapt to any dialect in a few hours or days.

The Categories of SQL Commands (Sub-languages)

There are 5 main sub-languages within SQL,

- DDL (Data Definition Language)
- DML (Data Manipulation Language)
- DCL (Data Control Language)
- TCL (Transaction Control Language)
- DQL (Data Query Language)

Types of Database Management Systems (DBMS)

The most common type of database you will use in finance is the Relational Database Management System (RDBMS). These are the strict, table-based systems we discussed earlier. The "Big Four" players here are Microsoft SQL Server, Oracle Database, MySQL, and PostgreSQL. There is also SQLite, a tiny, file-based database that lives inside your phone or web browser to store simple local data.

What is a Database Management System?

Database Management System (DBMS), structural or relational databases, non-structural relational database and data, examples.

A database management system (DBMS) is a set of programs used to define, administer, and process databases and their associated applications. The database being managed is, in essence, a structure that you build to hold valuable data. A DBMS is the tool you use to build that structure and operate on the data contained within the database.

Figure 1-1 shows that the user communicates with the database through the DBMS. The DBMS masks the physical details of the database storage so that the application need only concern itself with the logical characteristics of the data, not with how the data is stored.

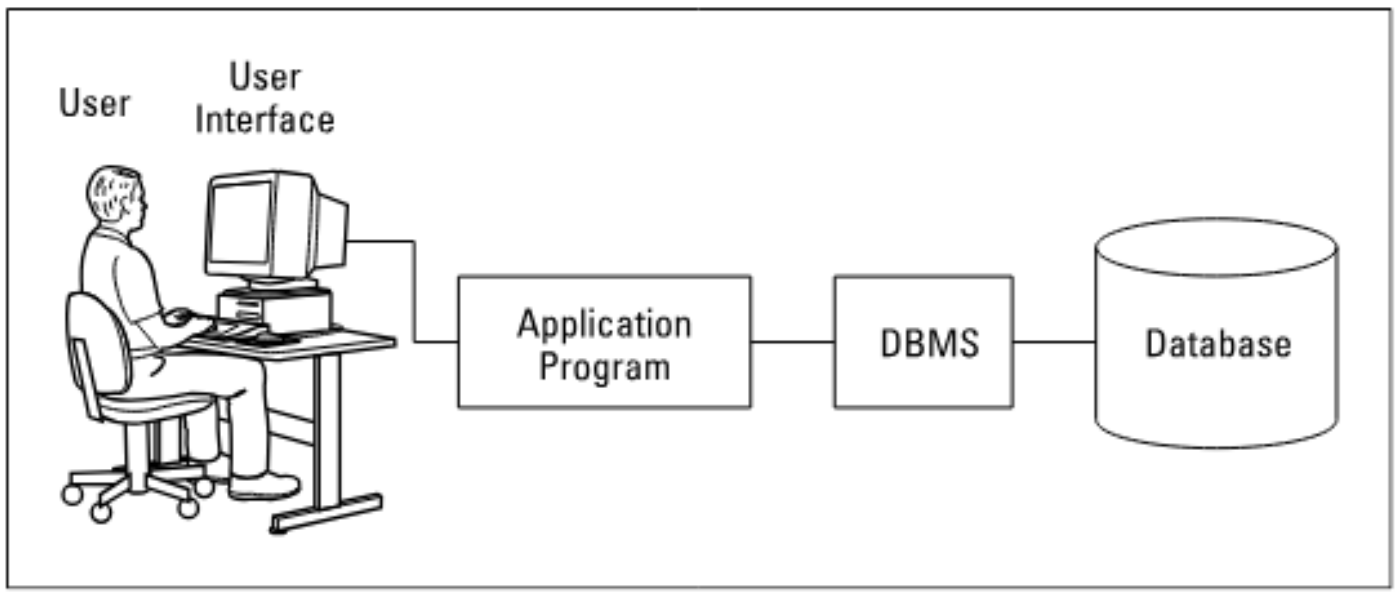


FIGURE 1-1: A block diagram of a DBMS-based information system.

Image retrieved from "SQL for Dummies, Allen G. Talyor (2018, Wiley), 9th Edition."

What is data (What is relational and non-relational data)?

Data is simply recorded information, but how we organize that information changes how we use it.

Relational Data is the type you will encounter most often in finance, accounting, and operational business systems. It is highly organized and structured, much like a neatly kept filing cabinet or a collection of Excel spreadsheets. In a relational database (like SQL Server or Oracle), data is stored in tables with fixed rows and columns. Every piece of data must follow strict rules: a "Date" column must contain a date, and a "Price" column must contain a number. These tables are linked, or "related" to each other using unique keys (like your Student ID). This structure ensures high accuracy and consistency, which is why it is the standard for banking systems where a missing decimal point could be a disaster.

Non-Relational & Cloud Databases

Non-Relational (NoSQL): Unlike strict SQL tables, Non-Relational databases are designed for the messy, unstructured data of the internet age. Imagine a laundry basket where you can throw in anything: PDFs documents, word documents, emails, or social media posts, without needing to fold or organize them first.

Common NoSQL Types:

- **Document Stores (e.g., MongoDB):** Stores data in flexible "JSON" documents (like a nested list). Great for product catalogs.
- **Key-Value Stores (e.g., Redis):** Extremely fast simple storage. Used for caching (remembering your login session).
- **Graph Databases (e.g., Neo4j):** Focuses on connections. Used for social networks and **fraud detection** (finding hidden links between criminals).

Cloud Data Warehouses: In the modern financial world, you will frequently hear about platforms like **Snowflake, Google BigQuery, and Amazon Redshift**. These are specialized **SQL** databases designed strictly for **analytics** (not daily transactions). They replace old-school servers because they separate "Storage" from "Computing Power." This allows banks to scan billions of rows in seconds and scale up their power only when running heavy end-of-month reports.

Summary Checklist

If the data looks like...	It is likely...	Example
A neat Grid	**Relational** (SQL)	Bank Transactions, Student Grades
A Tree / Nested List	**Document** (NoSQL)	Shopping Cart, User Profile
A Spiderweb	**Graph** (NoSQL)	Social Network, Fraud Ring
A Firehose	**Key-Value / Wide-Column**	Stock Ticker Feed, IoT Sensors

Feature	Relational (SQL)	Non-Relational (NoSQL)
Data Shape	**Tables** (Rows & Columns). Looks like a spreadsheet.	**Documents, Key-Values, Graphs**. Looks like a JSON file or a tree.
Flexibility	**Rigid.** You must define the structure (schema) *before* adding data. Changing it later is hard.	**Flexible.** You can add new fields on the fly without breaking the system.
Relationships	**Strong.** Tables are linked (e.g., Student	**Weak/None.** Data is often self-contained.

	ID connects to Grades table).	
Scaling	**Vertical.** To handle more data, you need a *bigger* computer (more RAM/CPU).	**Horizontal.** To handle more data, you just add *more* cheap computers (servers).
Transactions	**ACID Compliant.** Guaranteed accuracy. (If money leaves Account A, it *must* arrive in Account B).	**BASE Model.** "Eventually Consistent." (It's okay if a "Like" count is off by 1 for a few seconds).

Installation of SQL software or database management tool

There are many open-source database systems available, such as MySQL, PostgreSQL, and Microsoft SQL Server Management Studio. DBeaver is a *Database Management Tool* (GUI), not a database itself. It provides a universal interface that allows you to connect to and interact with various database systems, including MySQL and PostgreSQL.

Suggested SQL Textbooks: “SQL for Dummies” by Allen G. Taylor, and “Learning SQL” by Alan Beaulieu.

Suggested self-learning materials website: W3School SQL Tutorial, (Advanced) GeeksforGeeks SQL Tutorial.

Exercise: LeetCode, HackerRank (IBM, Capital One, Goldman Sachs), or ask Gen AI to create some questions

The Data Schema

A schema is essentially the blueprint or map of your database. It defines the structure of your data, including the relationships between tables, fields, and indexes. It defines the structure of your data, including the relationships between tables, fields, and indexes.

The Schema shows: "The `Orders` table is connected to the `Employees` table via the `employeeID` column. Here is an example of data schema for a Retail Sales DB. This schema represents a standard Transactional Sales System. It allows us to track *who* bought something (from Customers table), *what* they bought (from Products table), *who* sold it to them (from Employees table), and *when* it happened (from Orders table).

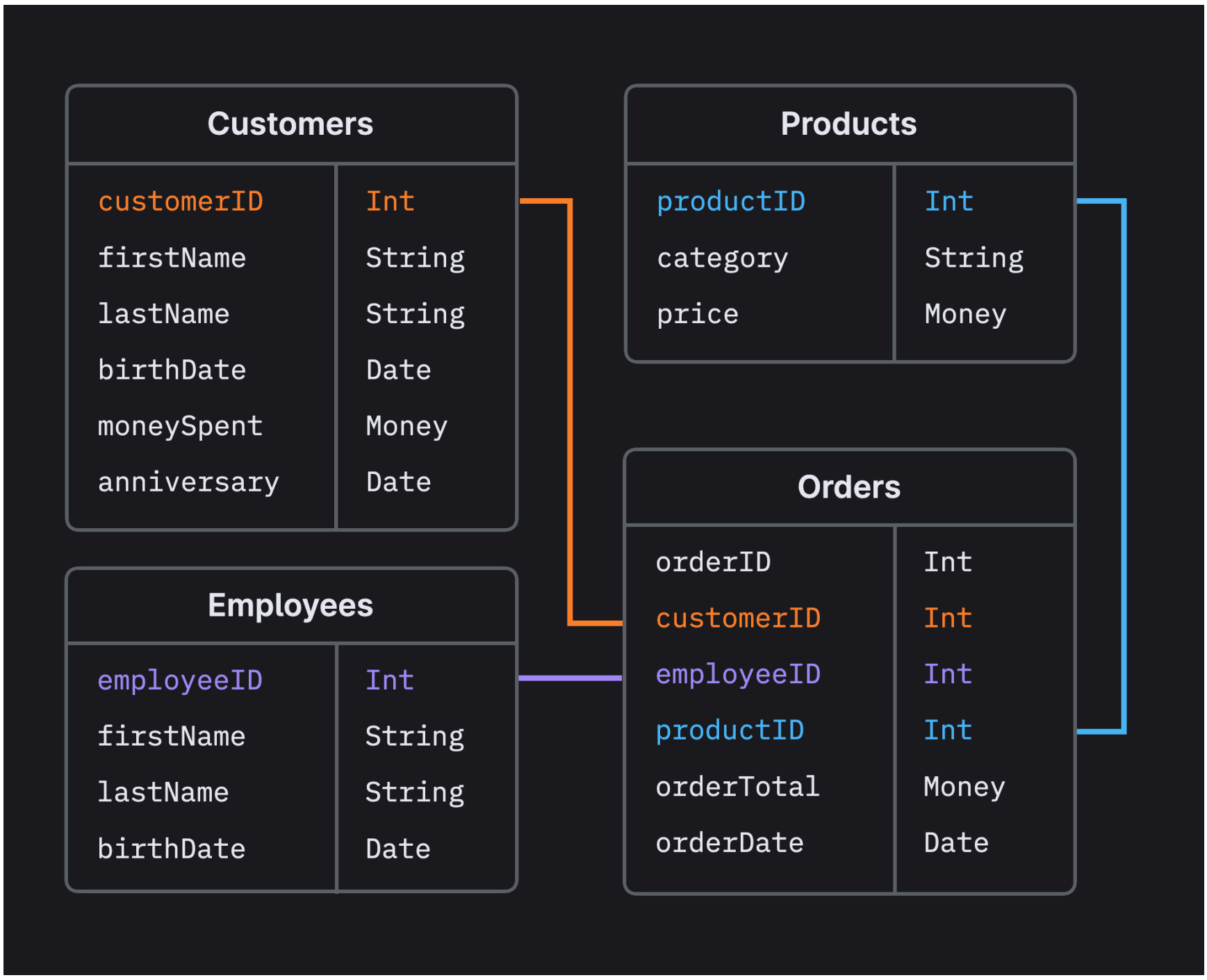
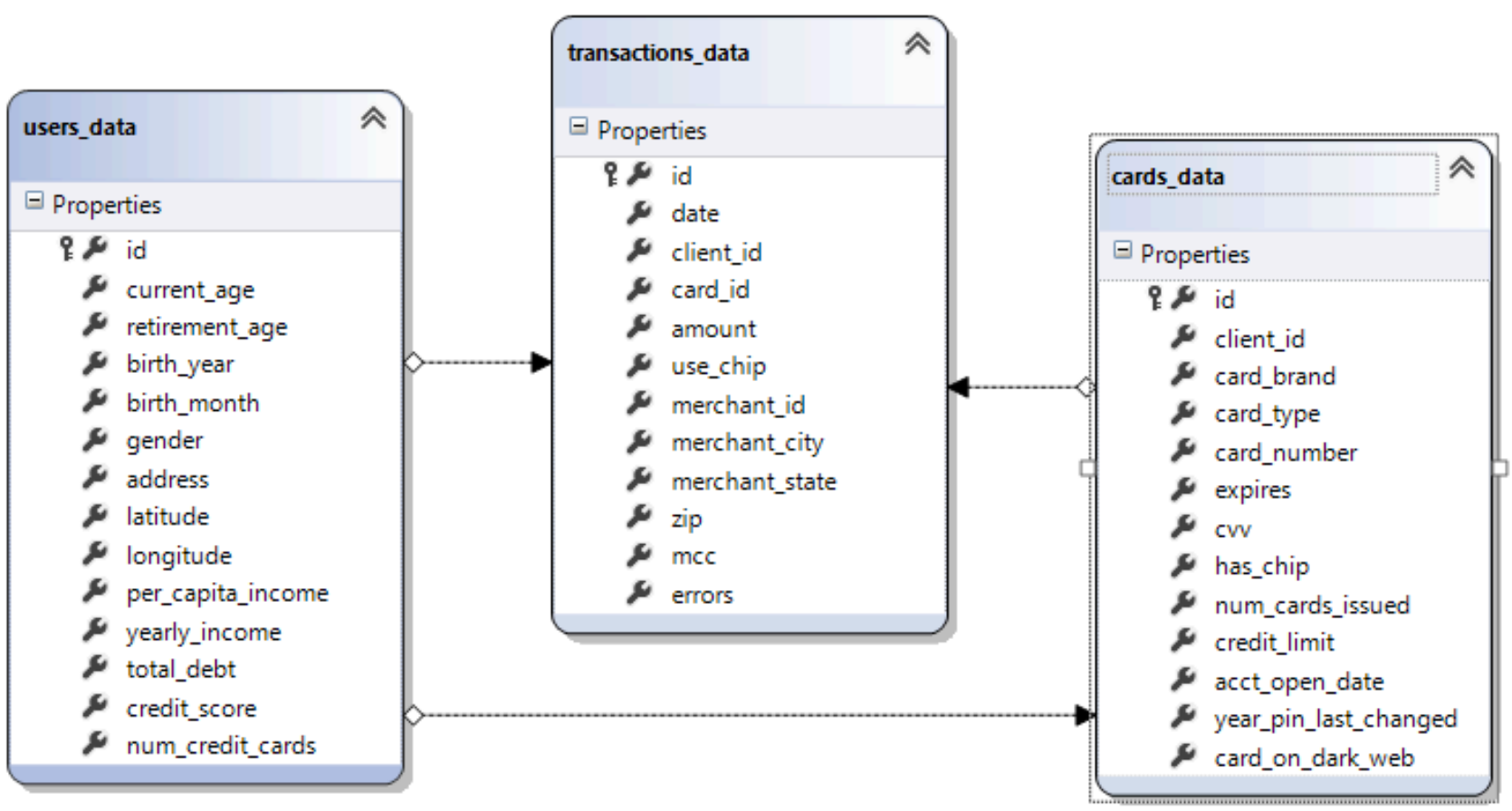


Image retrieved from schema-design-101-relational-databases.

Kaggle Financial Transactions Fraud Datasets

The dataset we use for this SQL workshop is retrieved from Kaggle called Financial Transactions Fraud Datasets: Kaggle-Transaction-Fraud-Datasets.

Now open the DBMS (`DBeaver`) on the lab computers, setup environment.



There are in total more than 13 million rows in `transactions_data` , this is why we need to use SQL to query data, rather than using Excel (max 1.024 million rows).

The `SELECT` statement is used to select data from a database.

```
# CTRL + ENTER will run the specific block of SQL code in DBbeaver.
PRAGMA table_info(table_name); # preview the data types for the table
PRAGMA table_info('Album');

# select all of the columns from the users_data table.
SELECT * FROM users_data; # note unlike python, SQL is NOT case-sensitive for keyword
s.
seleCT * FrOm Users_DAta;

SELECT id, current_age, birth_year, gender, yearly_income, total_debt FROM users_data;

SELECT column1, column2, ...
FROM table_name;

SELECT id, (total_debt / yearly_income) FROM users_data;
SELECT id, (total_debt / yearly_income) AS debt_to_income_ratio from users_data; # Not
e, just like Python, where you have to include underscores for naming.

SELECT * FROM users_data LIMIT 10; # select top 10
```

Aggregate Functions: Aggregate functions operate on a set or a group of values (usually a column) and return a single value. They are commonly used with the `GROUP BY` clause.

- `AVG()`
- `COUNT()`
- `MAX()`
- `MIN()`
- `SUM()`

String Functions: These functions are used to manipulate character data.

- `CONCAT()`
- `LENGTH()` / `LEN()`
- `UPPER()` / `UCASE()`
- `LOWER()` / `LCASE()`
- `SUBSTRING()` / `SUBSTR()` / `MID()`
- `REPLACE()`
- `TRIM()` / `LTRIM()` / `RTRIM()`

Numeric/Mathematical Functions: These functions perform mathematical operations.

- `ABS()`
- `ROUND()`
- `CEIL()` / `CEILING()`
- `FLOOR()`
- `SQRT()`
- `RAND()` / `RANDOM()`

For a complete list of functions supported by a specific database system, refer to its official documentation, such as the Microsoft Learn SQL documentation or the SQLite Built-In Scalar SQL Functions documentation.

Aggregate functions operate on a set or a group of values (usually a column) and return a single value.

- `AVG()` `COUNT()` `MAX()` `MIN()` `SUM()` `FIRST()` `LAST()`

```
-- Let's start with the simplest of the three datasets we are covering today: users_da
ta.
SELECT id, MAX(current_age)
FROM users_data;

SELECT AVG(current_age)
FROM users_data;

SELECT AVG(credit_score), MAX(total_debt)
FROM users_data;

SELECT
        AVG(credit_score) AS average_credit_score,
        MAX(total_debt) AS Maximium_total_debt
FROM users_data;

SELECT count(*)
FROM users_data;

SELECT
        AVG(yearly_income) AS average_income,
        SUM(yearly_income)/count(*) AS Average_yearly_income
FROM users_data;

SELECT * FROM users_data LIMIT 10; # select top 10
SELECT * FROM users_data LIMIT 10 OFFSET 1990; # select the last 10 rows within the da
taset. 2000 rows
```

How do you handle duplicate values or NULL values in your dataset?

These questions are often to ask during interviews. `count(*)` will count all of the rows in the table, however, it will also include NULL values.

```
-- The 'id' column is the Primary Key, as we mentioned every ID is unique and no dupli
cates are allowed.
SELECT COUNT(id)
FROM users_data;

SELECT COUNT(DISTINCT id)
FROM users_data;

SELECT gender
FROM users_data;

SELECT DISTINCT gender
FROM users_data;

SELECT COUNT(gender)
FROM users_data;

SELECT COUNT(DISTINCT gender)
FROM users_data;
```

Numeric/Mathematical Functions:

- `ABS()` `ROUND()` `CEILING()` `FLOOR()` `SQRT()` `RANDOM()`

```
-- Note: Built-in mathematical functions can only be applied to columns with numeric d
ata types.
-- 1. THE ABS() FUNCTION: Absolute Value
SELECT ABS(*)
FROM users_data; # will return error

SELECT ABS(Longitude)
FROM users_data;

-- 2. THE ROUND() FUNCTION
SELECT ROUND(Longitude,1)
FROM users_data;

-- 3. THE CEILING() FUNCTION: Always Round Up
SELECT CEILING(Longitude)
FROM users_data;
SELECT CEILING(1000.1);

-- 4. THE FLOOR() FUNCTION: Always Round Down
SELECT FLOOR(1000.7);

-- 5. THE SQRT() FUNCTION: Square Root
SELECT SQRT(10000);

-- 6. THE RANDOM() FUNCTION
SELECT RANDOM();

SELECT
    ABS(RANDOM()) % 10 + 1 AS Random_Number
FROM transactions_data
LIMIT 5;
```

String Functions: These functions are used to manipulate character data.

- `CONCAT()` `LENGTH()` `UPPER()` `LOWER()` `SUBSTR()` `TRIM()` / `LTRIM()` / `RTRIM()` `CAST()` `COALESCE()`

```
SELECT CONCAT('A', 'B', 'c')

SELECT CONCAT(address,' ,', latitude, ' ,', longitude) AS address_with_coordinates # y
ou can have 'address with coordinates'
FROM users_data;

SELECT
    LENGTH(CONCAT(address, ' ,', latitude, ' ,', longitude)) AS Length_of_strings
FROM users_data;

SELECT UPPER('address');

SELECT UPPER(CONCAT(address,' ,', latitude, ' ,', longitude)) AS 'address with coordin
ates'
FROM users_data;

SELECT LOWER(CONCAT(address,' ,', latitude, ' ,', longitude)) AS 'address with coordin
ates'
FROM users_data;

SELECT SUBSTR(address, 1, 5)
FROM users_data;
```

```
SELECT per_capita_income, sum(CAST(REPLACE(REPLACE(per_capita_income, '$', ''), ',',
'') AS int))
FROM users_data;
```

`WHERE` clause

The `WHERE` clause is used to filter records with comparison operators. It is used to extract only those records that fulfill a specified condition.

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

```
-- Comparison Operators: =, >, <, >=, <=, <> or !=, BETWEEN, LIKE, IN

-- Let's query individuals who are strictly under the age of 20.
SELECT *
FROM users_data
WHERE current_age < 20;

-- We can combine multiple conditions using AND.
-- Find individuals under 30 who also make more than $80k a year.
SELECT *
FROM users_data
WHERE current_age < 30 AND yearly_income > 60000; # use <= as well

-- Pull out the exact information for one specific customer using their ID.
SELECT *
FROM users_data
WHERE id = 1556;

PRAGMA table_info(transactions_data); # Recalled how to preview datatype of your tabl
e.
SELECT *
FROM transactions_data;
```

Wildcards

A wildcard character is used to substitute one or more characters in a string. Wildcard characters are used with the `LIKE` operator. The `LIKE` operator is used in a `WHERE` clause to search for a specified pattern in a column.

Symbol	Description
%	Represents zero or more characters
_	Represents a single character

```
SELECT *
FROM transactions_data
WHERE merchant_city = "Toronto";

SELECT *
FROM transactions_data
```

```
WHERE merchant_city LIKE "T%"; # String start with T, where % is a wildcard symbol

# Select all transactions where the merchant city starts with 'T' and contains the pat
tern 'oro', and the location is either Canada or New Jersey.
SELECT *
FROM transactions_data
WHERE merchant_city LIKE "T%" AND merchant_city LIKE "%oro%"
AND (merchant_state = 'Canada' or merchant_state = 'NJ');

# The IN operator allows you to specify multiple values in a WHERE clause.
SELECT merchant_city
FROM Transactions_data
WHERE merchant_city in ('Toronto', 'Mexico', 'Vancouver');

-- You can also use IN with a subquery!
SELECT * FROM transactions_data
WHERE merchant_city IN (SELECT merchant_city FROM transactions_data);

-- Let's bring back our REPLACE() function to clean the money column first.
SELECT amount, CAST(REPLACE(amount, '$', '') AS DECIMAL) AS Transaction_amount
FROM transactions_data
WHERE Transaction_amount BETWEEN 10 AND 12;
-- This does the exact same thing using standard operators.
SELECT amount, CAST(REPLACE(amount, '$', '') AS DECIMAL) AS Transaction_amount
FROM transactions_data
WHERE 10 < Transaction_amount AND Transaction_amount< 12;

-- We can filter text alphabetically!
SELECT merchant_city FROM transactions_data
WHERE merchant_city BETWEEN 'A' AND 'B';

SELECT *
FROM transactions_data
WHERE date BETWEEN '2010-07-01' AND '2010-07-31';
```

The SQL `ORDER BY`

The `ORDER BY` keyword is used to sort the result-set in ascending or descending order. The `ORDER BY` keyword sorts the records in ascending order by default. To sort the records in descending order, use the `DESC` keyword.

```
-- Next, we want to cover the ORDER BY clausem which is used to sort your result set.
-- By default, if you don't specify a direction, it always sorts in ASCENDING order (l
owest to highest, or 1, 2, 3...).
SELECT *
FROM users_data
ORDER BY id; -- or ORDER BY id ASC

SELECT *
FROM users_data
ORDER BY id DESC;

SELECT *
FROM users_data
ORDER BY gender;

SELECT *
FROM transactions_data
ORDER BY merchant_city, id;

SELECT *
```

```
FROM transactions_data
ORDER BY merchant_city ASC, id DESC;
```

The SQL GROUP BY Statement

The `GROUP BY` statement groups rows that have the same values into summary rows, like "find the number of transaction in each city". The `GROUP BY` statement is often used with aggregate functions (`COUNT()`, `SUM()`, `AVG()`) to group the result-set by one or more columns.

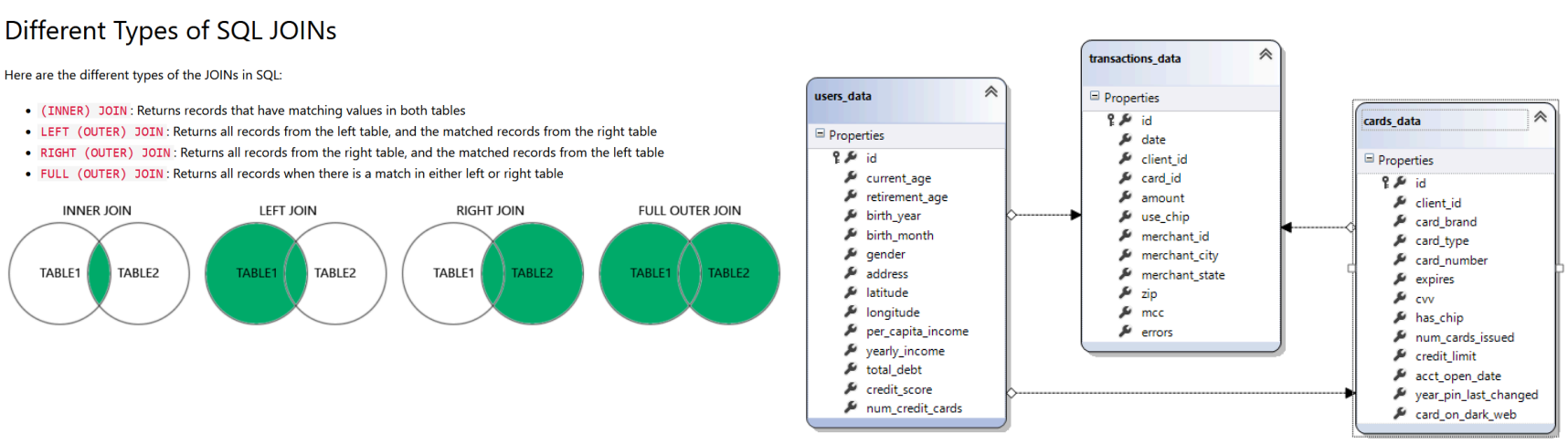
```
-- ==========================================
-- THE SQL BLUEPRINT
-- ==========================================
-- When building complex queries, you must write the clauses in this exact order:
SELECT column_name(s)
FROM table_name
WHERE condition              -- Filters raw rows BEFORE grouping
GROUP BY column_name(s)      -- Squishes the data into categories
ORDER BY column_name(s);     -- Sorts the final output
```

The `HAVING` clause was added to SQL because the `WHERE` keyword cannot be used with aggregate functions.

```
-- ==========================================
-- THE SQL BLUEPRINT
-- ==========================================
-- When building complex queries, you must write the clauses in this exact order:
SELECT column_name(s)
FROM table_name
WHERE condition              -- Filters raw rows BEFORE grouping
GROUP BY column_name(s)      -- Squishes the data into categories
HAVING condition             -- Filters the summary metrics AFTER grouping
ORDER BY column_name(s);     -- Sorts the final output
```

`JOIN` and `ON`

Now, we are going to learn one of the most important clauses in SQL, which is `JOINs`. A `JOIN` clause is used to combine rows from two or more tables, based on a related column between them.




```
SELECT [column_name]
FROM [tableA_name ]
JOIN [tableB_name] ON [tableA_name].[column_name] = [tableB_name].[column_name]

SELECT ud.id AS client_id, ud.current_age, ud.birth_year, ud.address, ud.per_capita_in
come _capita_income, cd.card_type, cd.card_number
FROM users_data ud
INNER JOIN cards_data cd ON ud.id = cd.client_id;

-- Not all clients have made transactions within our dataset's timeframe.
```

```
SELECT COUNT(DISTINCT client_id) FROM transactions_data;
SELECT COUNT(DISTINCT card_id) FROM transactions_data;

SELECT *
FROM users_data ud
JOIN cards_data cd ON ud.id = cd.client_id      -- Link User to their Cards
JOIN transactions_data td ON cd.id = td.card_id; -- Link Card to its SPECIFIC Transact
ions

SELECT *
FROM transactions_data t
LEFT JOIN cards_data c ON c.id = t.card_id
GROUP BY card_id;

-- Let's combine JOINs with our GROUP BY skills! We want to see the total amount spent
on each specific card. (Notice we use SUM to add up the money, and COUNT to find the n
umber of swipes).
SELECT t.client_id, card_id, SUM(CAST(REPLACE(t.amount, "$", "") AS DECIMAL)) AS total
 amount, c.card_number, count(c.card_number) AS Number_of_transactions
FROM transactions_data t
LEFT JOIN cards_data c ON c.id = t.card_id
GROUP BY c.card_number;
```

Full Example:

```
# how to combine row-level filtering with aggregate filtering. It answers a specific b
usiness question: "Which users over the age of 35 are using a shared credit card?"
SELECT
    u.id,
    u.current_age,
    COUNT(c.card_number) AS total_cards
FROM users_data u
JOIN cards_data c ON u.id = c.client_id
WHERE u.current_age > 35
GROUP BY u.id
HAVING total_cards > 1
ORDER BY total_cards DESC;
```

You may think the SQL query process is top-to-bottom, exactly as you write it: starting with `SELECT` and ending with `LIMIT`. Many SQL learning website, or textbooks may not teach you this. The database engine actually processes the query almost completely backward! This is one of the most important concepts for beginners to grasp. Understanding the difference between the **Lexical Order** (how we type it) and the **Logical Order** (how the engine executes it) explains why certain errors happen—like why you can't use an alias defined in the `SELECT` clause inside your `WHERE` clause.

Process order: (FROM, JOIN, ON) → (WHERE) → (GROUP BY) → (HAVING) → (SELECT) → (DISTINCT) → (ORDER BY) → (OFFSET) → (LIMIT)

```
-- We can connect all three of our tables together to get a complete picture of the bu
siness.
-- Notice how the 'cards_data' table acts as the perfect "bridge" between the transact
ion and the user!
SELECT *
FROM transactions_data T
JOIN cards_data C ON T.card_id = C.id
JOIN users_data U ON C.client_id = U.id;
```

Summary

```
SELECT
    DISTINCT [column_names_with AS [nickname/rename]
FROM
    [table_name_or_database_name]
JOIN [other_table_name] ON []
WHERE
    [conditions for_filltering numbers/strings/categories, uses Mathematical calculati
on symbols]
GROUP BY
    [column_name]
HAVING
    [additional conditions follow by groupby, constants, strings, Window Functions(av
g,...), column_name]
ORDER BY
    [column_name1 DESC, column_name2 ASC column3 DESC]
LIMIT 3 OFFSET 2; [start_from_third_row_and_limit_to_show_3_rows_only]
```

Process order: (FROM, JOIN, ON) → (WHERE) → (GROUP BY) → (HAVING) → (SELECT) → (DISTINCT) → (ORDER BY) → (OFFSET) → (LIMIT)

1. COUNT(*) and COUNT(1) differences?

This is one of the most common questions students ask, and it is surrounded by one of the biggest myths in the SQL world! Here is the straightforward answer you can share with your class: **Functionally and for performance, there is absolutely zero difference between `COUNT(*)` and `COUNT(1)` in modern databases.**

Exercise (if we have time, we can go onto HackerRank or LeetCode to practice with students)

https://www.hackerrank.com/domains/sql?filters%5Bstatus%5D%5B%5D=unsolved&badge_type=sql

https://leetcode.com/studyplan/top-sql-50/